How SpiceMCP compares

Three circuits, three ways of designing them, one independent checker. SpiceMCP 1.1.0, ngspice 42, 9 Nov 2026.

The short answer

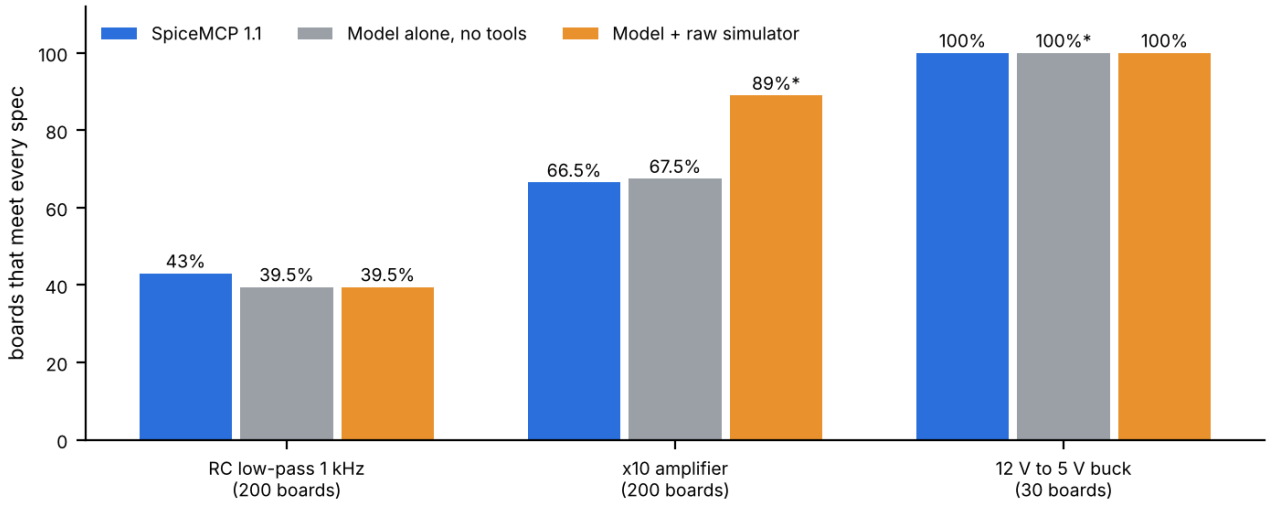
• **All three ways met every spec at nominal on all three circuits.** For simple, well-known circuits a strong model with no tools already gets the textbook answer. SpiceMCP did not beat it on pass rate.

• **Yield was a tie on the RC filter and the buck, and SpiceMCP lost on the amplifier** (66.5% vs 67.5% model alone vs 89% model + simulator). The 89% came from a 1.01 kΩ resistor, a value you cannot buy; with 1.00 kΩ it is 67.5%. A centred buyable pair (10.2 kΩ / 1.13 kΩ) reaches 91%, so SpiceMCP left 25 points on the table.

• **The RC filter caps out near 40% for everyone:** ±2% cutoff with 5% capacitors is not buildable reliably. SpiceMCP was the only arm that said so and named the fix (tighter capacitor); the plain model also flagged it in words.

• **Where SpiceMCP clearly wins is what you get back:** a measured yield, a parts list in buyable values, the ratings to buy each part at, a drawn schematic and a report. The other arms return a netlist and a claim.

• **Where it loses:** slowest (14 min for the buck, mostly the 90-run yield check) and about 35% more billed tokens than a model driving a raw simulator. Its smallest-parts goal also leaves thinner margins (buck ripple up to 44 of 50 mV vs 5 mV with bigger capacitors).



Yield: share of randomly built boards (resistors 1%, capacitors 5%, inductors 20%, supply ±5%) that meet every spec at −40, 25 and 85 °C. Same random boards for every arm. * = uses a value you cannot buy or an ideal switch.

What was compared

• **SpiceMCP 1.1:** the calls a model would make: list templates, start from the template, optimise 20 rounds, finish (standard values + yield), report, draw. Driven by a script over the real MCP connection, so its time has no model thinking in it.

• **Model alone, no tools:** a fresh model given only the spec text; it wrote the three netlists from memory, without simulating.

• **Model + raw simulator:** a fresh model allowed only to write netlists and run ngspice, with no templates, optimiser, value snapping or yield. Budget about 25 runs.

• **Textbook reference:** for the RC filter and the amplifier the textbook answer (15.8 kΩ with 10 nF; 9.09 kΩ over 1 kΩ) is exactly what the model alone wrote, so it is not a separate column.

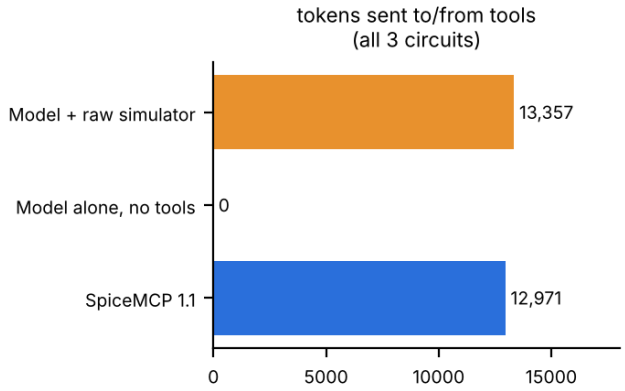
• **Other SPICE servers:** installing third-party packages was blocked in this sandbox, so they were compared on features from their public pages, not run.

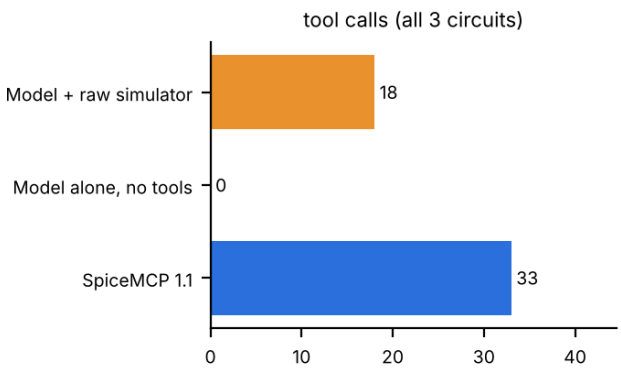
• **The checker** (measure/checker.py) is written separately from SpiceMCP, strips every arm's own analyses and measures all of them with the same decks.

Results, circuit by circuit

circuit	arm	values	meets spec (nominal)	measured	yield	all values buyable
RC low-pass 1 kHz	SpiceMCP 1.1	R1 88.7k, C1 1.8n	yes	fc 997 Hz	43% of 200	yes
RC low-pass 1 kHz	Model alone, no tools	R1 15.8k, C1 10n	yes	fc 1007 Hz	39.5% of 200	yes
RC low-pass 1 kHz	Model + raw simulator	R1 1.58k, C1 100n	yes	fc 1007 Hz	39.5% of 200	yes
x10 amplifier	SpiceMCP 1.1	R2 90.9k, R1 10k	yes	20.074 dB, BW 1.12 MHz, overshoot 0%	66.5% of 200	yes
x10 amplifier	Model alone, no tools	Rf 9.09k, Rg 1k	yes	20.072 dB, BW 1.10 MHz, overshoot 0%	67.5% of 200	yes
x10 amplifier	Model + raw simulator	Rf 9.09k, Rg 1.01k	yes	19.994 dB, BW 1.11 MHz, overshoot 0%	89% of 200	no (Rg)
12 V to 5 V buck	SpiceMCP 1.1	5 R/C/L	yes	11.4 V: 4.991 V / 32 mV, 12 V: 4.992 V / 35 mV, 12.6 V: 4.991 V / 35 mV	100% of 30	yes
12 V to 5 V buck	Model alone, no tools	9 R/C/L	yes	11.4 V: 5.000 V / 3 mV, 12 V: 5.000 V / 4 mV, 12.6 V: 5.000 V / 3 mV	100% of 30	yes; ideal switch
12 V to 5 V buck	Model + raw simulator	13 R/C/L	yes	11.4 V: 4.999 V / 11 mV, 12 V: 5.001 V / 15 mV, 12.6 V: 4.998 V / 14 mV	100% of 30	yes

Effort: time, calls and tokens





arm	wall-clock (3 circuits)	tool calls	simulator runs	tokens to/from tools	model output tokens	what you get back
SpiceMCP 1.1	13 min 53 s (RC 2 s, amp 3 s, buck 13.8 min), no model thinking included	33	about 1,100 (optimiser, stability and noise decks, yield boards at 3 tempera tures)	12,971	not measured (script played the model; its 33 requests are 1,005 tokens)	standard-value parts list (CSV), yield with confidence, part ratings to buy, schematic, report with plots
Model alone, no tools	61 s	0	0	0	11,736 (mostly reasoning)	3 netlists and one line of reasoning each
Model + raw simulator	3 min 18 s	18	37	13,357	30,304 (mostly reasoning)	3 netlists and its own corner table; no parts list, no yield, no drawing

Tokens counted with the same tokenizer for all arms (±25% against today's models). Billed input if nothing is cached, with each turn resending the conversation: SpiceMCP 197k (its 3,006-token tool list goes out every turn, and the buck needed 13 finish_design polls), model + raw simulator 145k, model alone about 1k. Model output tokens are from the API for the two model arms.

Against other open-source SPICE servers (features, from their public pages)

	SpiceMCP 1.1	ltspice-mcp 0.8 (Cognitohazard)	mcp-spice 0.1 (zesun33)	spicebridge
runs ngspice / LTspice	both	both	ngspice	ngspice
ready-made circuits sized by equations	13	no	no	not stated
optimises part values	yes	no (API usable by one)	no	not stated
snaps to buyable values	E12/E24/E96	no	no	not stated
Monte Carlo / yield	yes, with confidence bound	yes	no	listed
temperature and supply corners	yes	corners mentioned	no	worst-case listed
parts list and ratings to buy	yes	no	no	no
draws the schematic	yes, auto-placed	edits .asc	no	no
MCP tools	14	8	2	not checked

From each project's public page on 9 Nov 2026 (ltspice-mcp: PyPI page; mcp-spice: Glama listing; spicebridge: search listing only, its page did not load). Not run here: installing third-party packages was blocked in this sandbox. Treat 'not stated' as unknown, not as missing.

Brutally honest reading

• This benchmark does not show that SpiceMCP designs better circuits than a strong model. On these three it designs about the same circuits.

• Its value here is verification and paperwork: it measured yield over 30 to 200 boards at three temperatures and a ±5% supply, while the model-alone arm said 'meets spec' without checking and was right only at nominal.

• The model + raw simulator arm checked corners by itself, but hit a simulator convergence failure it could not fix, used a non-standard resistor, and spent 37 runs and 3 minutes doing what one SpiceMCP call does.

• The model-alone buck used an ideal switch, which is not a part you can buy and skips the gate drive. It still passed, so the checker does not punish idealisations; a stricter benchmark should.

• The baselines were handed SpiceMCP's own working op-amp and comparator models. That removes the hardest part of the buck for them, so the buck result flatters the baselines.

• The model used for the baselines is a strong one. SpiceMCP is meant to help weaker models; that case is untested here and is where the gap should be largest.

What to fix next

• **finish_design should centre ratios, not just snap each value:** search standard-value pairs for the one closest to the middle of the limits. On the amplifier that lifts yield from 66.5% to 91%.

• **Do not shrink parts to the edge by default:** the 'smallest capacitor / inductor' goals left the buck at 44 mV worst-case ripple against a 50 mV limit. Aim for the middle of the limit, or give the user the size-vs-margin choice.

• **Make long finishes cheaper to wait for:** 13 identical polling calls cost about 70k billed tokens on the buck. Return an estimated finish time and tell the model to wait that long.

• **Say 'not buildable at this tolerance' up front:** when the spec window is narrower than the part tolerance (RC ±2% with 5% capacitors), say so before optimising.

Limits of this benchmark

• Three circuits, one run each; model arms are not repeated, so luck is not averaged out.

• Simulation only (ngspice 42). The buck ripple needs a 2 ns time step to be honest: at 20 ns SpiceMCP's buck reads 180 mV. The raw-simulator buck only runs at 20 ns or 5 ns, so its ripple number is less certain.

• Buck yield is 30 boards per arm (90 runs at three temperatures): 30/30 means at least 90% with 95% confidence. RC and amplifier use 200 boards.

• Tokens are counted with an older public tokenizer and exclude the tool harness's own system prompt, which is the same for every arm.